Quadratic Assignment Problem

Definitions, NP-Hardness, Algorithms, and Experiments

Md. Jakaria Hossain (2005026)

Musa Tur Farazi (2005038)

Md. As-Aid Rahman (2005039)

MD Sajid Mostafiz Noor (2005051)

Tausif Rashid (2005052)

Awesh Islam (2005054)

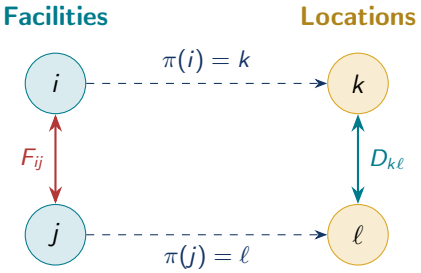
CSE462 — Algorithm Engineering Sessional

March 2026

Agenda

1. **Problem Definition** — Koopmans–Beckmann formulation & variants
2. **Complexity Theory** — NP-Hardness via two reductions
3. **Algorithm Landscape** — Exact, approximation & metaheuristic families
4. **Chosen Algorithms** — Branch-and-Bound & Tabu Search
5. **Proposed Modifications** — Greedy UB, LB-Ordered DFS, Reactive Tenure, Diversification
6. **Experimental Results** — Runtime, solution quality, and scalability

Motivation — Core Idea



Key quantities:

- F_{ij}: flow between facilities i and j
- $D_{k\ell}$: distance between locations k and ℓ
- π: assignment (bijection) of facilities to locations

Total Cost

$$\text{Cost}(\pi) = \sum_{i=1}^{n} \sum_{j=1}^{n} F_{ij} \cdot D_{\pi(i),\pi(j)}$$

Minimize the total weighted distance!

Formal Definition (Koopmans–Beckmann, 1957)

Given:

- Flow matrix $F \in \mathbb{R}^{n \times n}$, Distance matrix $D \in \mathbb{R}^{n \times n}$
- Optionally, linear cost matrix $C \in \mathbb{R}^{n \times n}$

Objective

$$\min_{\pi \in S_n} \underbrace{\sum_{i=1}^{n} \sum_{j=1}^{n} F_{ij} \, D_{\pi(i),\pi(j)}}_{\text{Quadratic Pair Cost}} + \underbrace{\sum_{i=1}^{n} C_{i,\pi(i)}}_{\text{Linear Cost}}$$

- S_n: set of all permutations of $\{1, \ldots, n\}$ $\quad (|S_n| = n!)$
- When $C = 0$: Homogeneous QAP

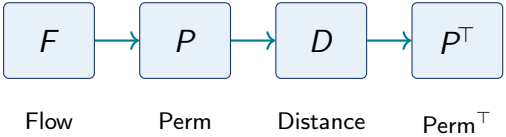
Koopmans & Beckmann, *Econometrica*, 1957

Trace Formulation

Let Π_n be the set of all $n \times n$ permutation matrices.

Trace Form

$$\min_{P \in \Pi_n} \quad \mathrm{tr}\big(F\,P\,D\,P^\top\big) \;+\; \mathrm{tr}\big(C^\top P\big)$$



Equivalent to Koopmans–Beckmann when P encodes π.

0–1 Quadratic Integer Program

Binary variable: $x_{ik} \in \{0, 1\}$: facility i assigned to location k.

Assignment Constraints

$$\sum_{k=1}^{n} x_{ik} = 1 \quad \forall\, i \qquad\qquad \sum_{i=1}^{n} x_{ik} = 1 \quad \forall\, k$$

Objective

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{k=1}^{n} \sum_{\ell=1}^{n} F_{ij}\, D_{k\ell}\, x_{ik}\, x_{j\ell} + \sum_{i=1}^{n} \sum_{k=1}^{n} C_{ik}\, x_{ik}$$

- n^2 binary variables, $2n$ equality constraints
- Quadratic terms $x_{ik}\, x_{j\ell}$ make this non-convex

QAP Decision Problem (QAP-DEC)

Instance

Matrices $F, D \in \mathbb{R}^{n \times n}$ and a cost bound $B \in \mathbb{R}$.

Question

Does there exist $\pi \in S_n$ such that $\mathrm{Cost}(\pi) \leq B$?

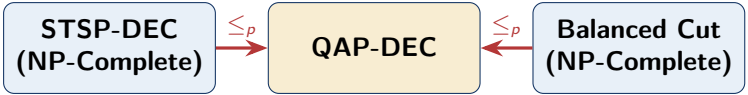
Lemma: QAP-DEC $\in$ NP

Certificate: A permutation π.
Verification: Compute $\mathrm{Cost}(\pi) = \sum_i \sum_j F_{ij} D_{\pi(i),\pi(j)}$ in $\mathcal{O}(n^2)$ time and check $\mathrm{Cost}(\pi) \leq B$.

Complexity Overview

Theorem

QAP-DEC is NP-Complete. Therefore the optimization QAP is NP-Hard.



Recall: Symmetric Travelling Salesman Problem (STSP)

Given cities with symmetric distances, find a tour of cost $\leq L$. Sahni & Gonzalez, *JACM*, 1976

Reduction 1: STSP $\leq_p$ QAP

Given: STSP instance with distance matrix $D^{\text{TSP}} \in \mathbb{R}^{n \times n}$ and bound L.

Construction (polynomial time)

1. Set $D := D^{\text{TSP}}$ (same distance matrix)
2. Set $B := 2L$ (QAP cost bound)
3. Construct F as the adjacency matrix of an n-cycle

- Reduction runs in $\mathcal{O}(n^2)$ time
- F has exactly $2n$ ones (each vertex connected to two neighbors)
- Key idea: the QAP cost with cycle flow counts each tour edge twice

Sahni & Gonzalez, 1976

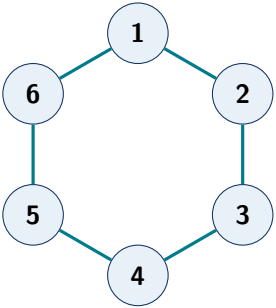
Constructing the Cycle Flow Matrix F

$$F_{ij} = \begin{cases} 1 & \text{if } |i-j| = 1 \text{ or } \{i,j\} = \{1,n\} \\ 0 & \text{otherwise} \end{cases}$$

Example ($n = 6$):

$$F = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Cycle graph:

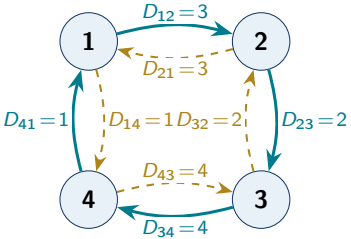


Key Lemma

Lemma

For every permutation $\pi \in S_n$: $C_{\text{QAP}}(\pi) = 2 \cdot \mathrm{TourLen}(\pi)$

Visualized ($n=4$, $\pi = \mathrm{id}$):



Forward + backward on each cycle edge

$C_{\text{QAP}} = (3+3)+(2+2)+(4+4)+(1+1) = 20 = 2\times 10$

Proof sketch:

$$
\begin{aligned}
C_{\text{QAP}}(\pi) &= \sum_i \sum_j F_{ij}\, D_{\pi(i),\pi(j)} \\
&= \sum_{\{i,j\}:\, F_{ij}=1} \left(D_{\pi(i),\pi(j)} + D_{\pi(j),\pi(i)}\right) \\
&= 2 \sum_{e \in \text{cycle}} D_{\pi(\text{endpoints of } e)} \\
&= 2 \cdot \mathrm{TourLen}(\pi)
\end{aligned}
$$

Uses symmetry $D_{kl} = D_{lk}$ and the fact that each cycle edge contributes a forward *and* backward term.

Correctness of Reduction 1

Forward direction: STSP YES $\Rightarrow$ QAP YES

If π^* is a tour with $\mathrm{TourLen}(\pi^*) \leq L$, then

$$C_{\mathrm{QAP}}(\pi^*) = 2 \cdot \mathrm{TourLen}(\pi^*) \leq 2L = B \quad \checkmark$$

Backward direction: QAP YES $\Rightarrow$ STSP YES

If π satisfies $C_{\mathrm{QAP}}(\pi) \leq B = 2L$, then

$$\mathrm{TourLen}(\pi) = \tfrac{1}{2}\, C_{\mathrm{QAP}}(\pi) \leq L \quad \checkmark$$

Result

Since STSP-DEC is NP-Complete and STSP $\leq_p$ QAP, we conclude **QAP-DEC is NP-Hard**.

Reduction 2: Balanced Cut $\leq_p$ QAP

Balanced Cut (Fixed-Size Cut): Given graph $G = (V, E, w)$ and integer K, is there a partition $(S, V \setminus S)$ with $|S| = \lfloor n/2 \rfloor$ and $\mathrm{cut}(S) \leq K$?

Construction

- $F_{uv} := w_{uv}$ (edge weights become flows)
- Locations split into two groups A and B, each of size $\lfloor n/2 \rfloor$
- Distance matrix:

$$D_{k\ell} = \begin{cases} 0 & \text{if } k, \ell \text{ in same group} \\ 1 & \text{if } k, \ell \text{ in different groups} \end{cases}$$

- QAP bound $B := 2K$

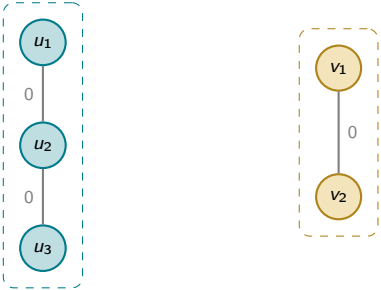
Garey & Johnson, 1979

Balanced Cut: Lemma & Proof

Key Lemma

$$C_{\mathrm{QAP}}(\pi) = 2 \cdot \mathrm{Cut}\big(S(\pi)\big) \qquad \text{where } S(\pi) = \{u : \pi(u) \in A\}$$

Proof:



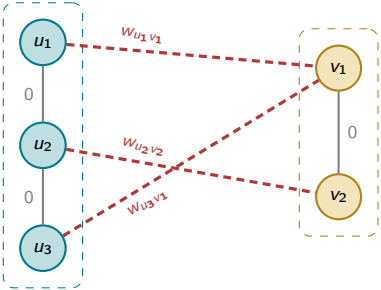
Balanced Cut: Lemma & Proof

Key Lemma

$$C_{\text{QAP}}(\pi) = 2 \cdot \text{Cut}\big(S(\pi)\big) \qquad \text{where } S(\pi) = \{u : \pi(u) \in A\}$$

Proof:

- **Step 1:** Each term $F_{uv}\, D_{\pi(u),\pi(v)} = w_{uv} \cdot 1[\text{different groups}]$



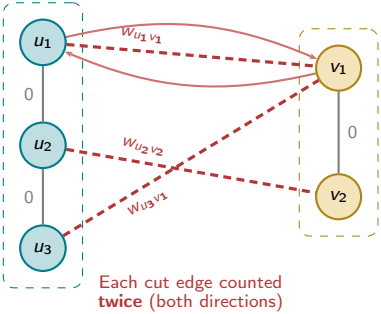
Balanced Cut: Lemma & Proof

Key Lemma

$$C_{\text{QAP}}(\pi) = 2 \cdot \text{Cut}\big(S(\pi)\big) \qquad \text{where } S(\pi) = \{u : \pi(u) \in A\}$$

Proof:

- **Step 1:** Each term $F_{uv}\, D_{\pi(u),\pi(v)} = w_{uv} \cdot 1[\text{different groups}]$
- **Step 2:** Sum over all ordered pairs — each cut edge (u, v) counted once as (u, v) and once as (v, u).



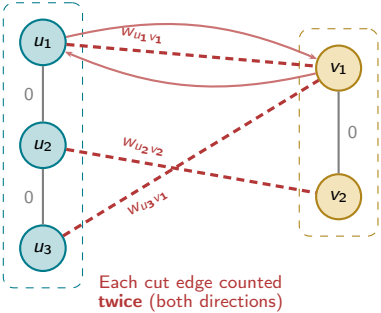
Balanced Cut: Lemma & Proof

Key Lemma

$$C_{\text{QAP}}(\pi) = 2 \cdot \text{Cut}\big(S(\pi)\big) \qquad \text{where } S(\pi) = \{u : \pi(u) \in A\}$$

Proof:

- **Step 1:** Each term $F_{uv}\, D_{\pi(u),\pi(v)} = w_{uv} \cdot 1[\text{different groups}]$
- **Step 2:** Sum over all ordered pairs — each cut edge (u, v) counted once as (u, v) and once as (v, u).
- **Step 3:** Therefore $C_{\text{QAP}}(\pi) = 2 \cdot \text{Cut}\big(S(\pi)\big)$. ■



Balanced Cut: Conclusion

Reduction Equivalence

Balanced Cut: Conclusion

Reduction Equivalence

- **Forward:** $\mathrm{Cut}(S) \leq K \implies C_{\mathrm{QAP}} = 2\,\mathrm{Cut}(S) \leq 2K = B$

Balanced Cut: Conclusion

Reduction Equivalence

- **Forward:** $\mathrm{Cut}(S) \leq K \implies C_{\mathrm{QAP}} = 2\,\mathrm{Cut}(S) \leq 2K = B$
- **Backward:** $C_{\mathrm{QAP}} \leq 2K \implies \mathrm{Cut}(S) = \frac{1}{2}\,C_{\mathrm{QAP}} \leq K$

Balanced Cut: Conclusion

Reduction Equivalence

- **Forward:** $\mathrm{Cut}(S) \leq K \implies C_{\mathrm{QAP}} = 2\,\mathrm{Cut}(S) \leq 2K = B$
- **Backward:** $C_{\mathrm{QAP}} \leq 2K \implies \mathrm{Cut}(S) = \frac{1}{2}\,C_{\mathrm{QAP}} \leq K$

Balanced Cut: Conclusion

Reduction Equivalence

- **Forward:** $\mathrm{Cut}(S) \leq K \implies C_{\mathrm{QAP}} = 2\,\mathrm{Cut}(S) \leq 2K = B$
- **Backward:** $C_{\mathrm{QAP}} \leq 2K \implies \mathrm{Cut}(S) = \frac{1}{2}\,C_{\mathrm{QAP}} \leq K$

Takeaway

QAP faithfully encodes balanced graph partitioning — the optimal QAP assignment *is* the optimal balanced cut, confirming NP-hardness.

Existing Algorithms

Exact

Branch-and-Bound: Enumerate partial assignments, prune via lower bounds.

Gilmore, *SIAM J.*, 1962; Lawler, *Mgmt. Sci.*, 1963

Worst-case $\Omega(n!)$

Metaheuristic (Our Focus)

Tabu Search: Local search with memory to escape local optima.

Taillard, *Parallel Computing*, 1991

Simulated Annealing: Accept worse solutions with decreasing probability.

Burkard & Rendl, *EJOR*, 1984

Genetic Algorithms: Population-based evolution with crossover.

Drezner, *INFORMS J. Comp.*, 2003

Approximation

LP/SDP Relaxation: Relax integrality constraints, then round.

Spectral Bounds: Eigenvalue-based lower bounds.

Rendl & Sotirov, *Math. Prog.*, 2007

General QAP is hard to approximate.

Randomized

Random Restart: Run local search from many random starts, keep best.

GRASP: Greedy randomized construction followed by local search.

Resende et al., *Discrete Appl. Math.*, 1996

Survey: Loiola et al., *European J. Operational Research*, 176(2), 657–690, 2007.

Algorithm 1: Branch-and-Bound

Search space: Tree of partial assignments

Node:

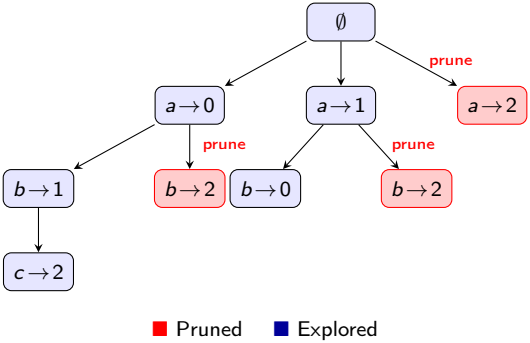
- Fixed set I_{fix}: assigned pairs
- Free set I_{free}: unassigned facilities

Branch:

- Assign next facility i to each free location k
- Creates one child per available location

Prune:

- Compute lower bound LB(node)
- If LB(node) $\geq$ UB $\Rightarrow$ cut branch



Gilmore–Lawler Lower Bound (GLB)

Cost decomposition at a partial assignment node:

$$C(\pi) = \underbrace{C_{\text{fixed}}}_{\text{known}} + \underbrace{C_{\text{fixed-free}}}_{\text{partially known}} + \underbrace{C_{\text{free-free}}}_{\text{unknown}}$$

Lower Bound via Linear Assignment

$$\mathrm{LB} = C_{\text{fixed}} + \min_{x} \sum_{i \in I_{\text{free}}} \sum_{k \in K_{\text{free}}} \hat{M}_{ik}\, x_{ik} \qquad \text{(LAP)}$$

- $\hat{M}_{ik}$: reduced cost of assigning free facility i to free location k
- Solved by the Hungarian Algorithm in $\mathcal{O}(n^3)$

Gilmore, 1962; Lawler, 1963

Branch-and-Bound & GLB — Pseudocode

B&B Main Loop

Initialize: $\mathrm{UB} \leftarrow +\infty$, $Q \leftarrow \{\text{Root}\}$

while $Q \neq \emptyset$ **do**

  Pop node with smallest LB from Q

  if $\mathrm{LB} \geq \mathrm{UB}$: **continue** // *Prune*

  if node is complete:

    Update UB, π^*

  else:

    Pick unassigned facility i

    for each free location k:

      Create child (fix $i \to k$)

      $\mathrm{LB}(\text{child}) \leftarrow \mathrm{GLB}(\text{child})$

      Push child to Q

return π^*

GLB(π) Calculation

$C_{\text{fixed}} \leftarrow$ cost of fixed assignments in π

for each free facility i:

  for each free location k:

    $c_1 \leftarrow$ cost of $i \to k$ with fixed

    $\vec{f} \leftarrow$ free flows from i (sort asc)

    $\vec{d} \leftarrow$ free dists from k (sort desc)

    $c_2 \leftarrow \vec{f} \cdot \vec{d}$ // $C_{\textit{free-free}}$

    $\hat{M}_{ik} \leftarrow c_1 + c_2$

$Z_{\text{LAP}} \leftarrow \text{Hungarian}(\hat{M})$

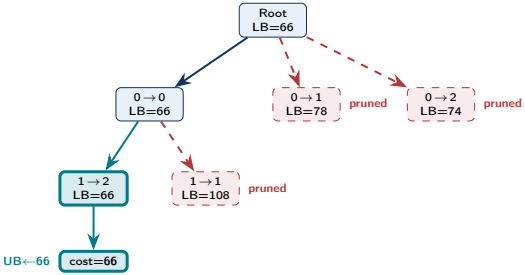
return $C_{\text{fixed}} + Z_{\text{LAP}}$

B&B Walkthrough: $n = 3$ Example

Instance:

$$F = \begin{pmatrix} 0 & 5 & 2 \\ 5 & 0 & 3 \\ 2 & 3 & 0 \end{pmatrix}$$

$$D = \begin{pmatrix} 0 & 8 & 1 \\ 8 & 0 & 4 \\ 1 & 4 & 0 \end{pmatrix}$$



1. Root: LB=66, UB=∞. Branch on facility 0.
2. Expand $0 \to 0$ (LB=66 $<$ UB). Child $1 \to 2$: LB=66, reach leaf $\pi = (0, 2, 1)$, cost=66. UB$\leftarrow$66.
3. Child $1 \to 1$: LB=108 $\geq$ 66 $\Rightarrow$ pruned.
4. $0 \to 1$: LB=78 $\geq$ 66 $\Rightarrow$ pruned. $0 \to 2$: LB=74 $\geq$ 66 $\Rightarrow$ pruned.

Result: Optimal $\pi^* = (0, 2, 1)$, cost **66**. Only 4 of 6 nodes explored.

Algorithm 2: Robust Tabu Search

Concept

Local Search enhanced with short-term memory (tabu list).

- **State space:** permutations S_n
- **Neighborhood:** 2-swap $\pi' = \pi \circ (r, s)$
- **Neighborhood size:** $\binom{n}{2} = \mathcal{O}(n^2)$ neighbors
- **Challenge:** evaluating each neighbor naïvely costs $\mathcal{O}(n^2)$
  $\Rightarrow$ total per iteration: $\mathcal{O}(n^4)$
- **Key insight:** $\mathcal{O}(n)$ delta evaluation using incremental formula

Taillard, *EJOR*, 1991

$\mathcal{O}(n)$ Delta Evaluation

Consider swapping positions r and s. Let $p = \pi(r)$, $q = \pi(s)$.

$$\Delta(r, s) = \Delta^{\text{ext}} + \Delta^{\text{int}}$$

External part — $\mathcal{O}(n)$

$$\Delta^{\text{ext}} = \sum_{\substack{k=1 \\ k \neq r,s}}^{n} \big(F_{rk} - F_{sk}\big)\big(D_{q,\pi(k)} - D_{p,\pi(k)}\big) + \big(F_{kr} - F_{ks}\big)\big(D_{\pi(k),q} - D_{\pi(k),p}\big)$$

Internal part — $\mathcal{O}(1)$

$$\Delta^{\text{int}} = (F_{rs} - F_{sr})(D_{qp} - D_{pq})$$

Result: Each swap evaluated in $\mathcal{O}(n) \Rightarrow$ full neighborhood in $\mathcal{O}(n^3)$.

Taillard, 1991

Tabu Search — Pseudocode

```
Initialize: random permutation π;  π* ← π
for t = 1 to T do
    BestSwap ← null
    for all pairs (r, s), r < s:
        Compute Δ(r, s) in O(n)
        Check: is (r, s) tabu? Aspiration criterion?
        if allowed and Δ < Δ_best:
            BestSwap ← (r, s)
    Apply BestSwap to π
    Update tabu list: mark (r, s) tabu for τ iterations
    if Cost(π) < Cost(π*):
        π* ← π
return π*
```

Aspiration: allow tabu move if it improves π^*.

Tabu Search Walkthrough: $n = 4$ Example

Instance: 4 facilities, 4 locations. Start: $\pi = (1, 2, 3, 4)$, Cost $= 340$, tenure $\tau = 2$.

$\pi = (1, 2, 3, 4)$
Cost $= 340$

Key observations:

- Iter 2: accepted worsening move $(+14)$ — best non-tabu option
- Escapes the local optimum at 298
- Iter 3: new global best 276

Tabu list dynamics:

- After swap(r, s): reverse forbidden for $\tau = 2$ iterations
- Prevents cycling back immediately
- $(1, 2)$ expired after iter 3
- $\star$ = new global best

Tabu Search Walkthrough: $n = 4$ Example

Instance: 4 facilities, 4 locations. Start: $\pi = (1, 2, 3, 4)$, Cost $= 340$, tenure $\tau = 2$.

$$\boxed{\begin{array}{c}\pi = (1,2,3,4)\\ \text{Cost} = 340\end{array}} \xrightarrow[\Delta = -42]{\text{swap}(1,2)} \boxed{\begin{array}{c}\pi = (2,1,3,4)\\ \text{Cost} = 298\ \star\end{array}}$$

tabu: $(1, 2)$

Key observations:

- Iter 2: accepted worsening move $(+14)$ — best non-tabu option
- Escapes the local optimum at 298
- Iter 3: new global best 276

Tabu list dynamics:

- After swap(r, s): reverse forbidden for $\tau = 2$ iterations
- Prevents cycling back immediately
- $(1, 2)$ expired after iter 3
- $\star$ = new global best

Tabu Search Walkthrough: $n = 4$ Example

Instance: 4 facilities, 4 locations. Start: $\pi = (1, 2, 3, 4)$, Cost $= 340$, tenure $\tau = 2$.

$$\boxed{\begin{array}{c}\pi = (1,2,3,4)\\ \text{Cost} = 340\end{array}} \xrightarrow[\Delta = -42]{\text{swap}(1,2)} \boxed{\begin{array}{c}\pi = (2,1,3,4)\\ \text{Cost} = 298\ \star\end{array}} \xrightarrow[\Delta = +14]{\text{swap}(2,3)} \boxed{\begin{array}{c}\pi = (2,3,1,4)\\ \text{Cost} = 312\end{array}}$$

tabu: $(1, 2)$ tabu: $(1, 2), (2, 3)$

Key observations:

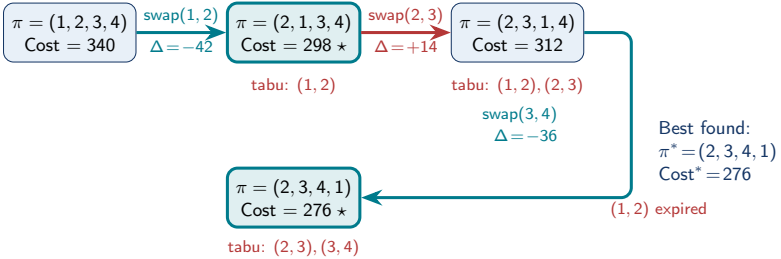
- Iter 2: accepted worsening move $(+14)$ — best non-tabu option
- Escapes the local optimum at 298
- Iter 3: new global best 276

Tabu list dynamics:

- After swap(r, s): reverse forbidden for $\tau = 2$ iterations
- Prevents cycling back immediately
- $(1, 2)$ expired after iter 3
- $\star$ = new global best

Tabu Search Walkthrough: $n = 4$ Example

Instance: 4 facilities, 4 locations. Start: $\pi = (1, 2, 3, 4)$, Cost $= 340$, tenure $\tau = 2$.



Key observations:

- Iter 2: accepted worsening move $(+14)$ — best non-tabu option
- Escapes the local optimum at 298
- Iter 3: new global best 276

Tabu list dynamics:

- After swap(r, s): reverse forbidden for $\tau = 2$ iterations
- Prevents cycling back immediately
- $(1, 2)$ expired after iter 3
- $\star$ = new global best

B&B Modification 1: Greedy Initial Upper Bound

Problem

Starting with $\mathrm{UB} = +\infty$ means no pruning in early iterations.

Solution: Greedy Heuristic

Before B&B, compute an initial solution:

1. Rank facilities by total flow: $\sum_j F_{ij}$ (descending)
2. Rank locations by total distance: $\sum_\ell D_{k\ell}$ (ascending)
3. Assign highest-flow facility to lowest-distance location

Effect: Tighter initial UB $\Rightarrow$ earlier pruning $\Rightarrow$ fewer nodes explored.

Loiola et al., *EJOR*, 2007

B&B Modification 2: LB-Ordered Child Exploration

Problem

DFS explores children in arbitrary order; a bad branch is expanded before a promising one.

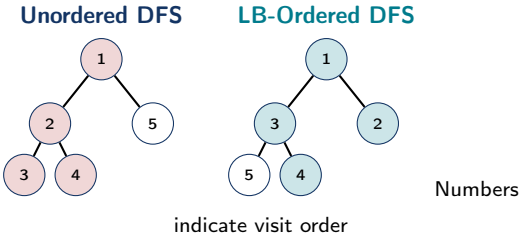
Solution

Before pushing children onto the DFS stack, sort by LB ascending.

Lowest-LB child is pushed *last* $\Rightarrow$ popped *first*.

Trade-off:

- \+ Steers DFS toward the optimum earlier
- \+ Same $\mathcal{O}(n)$ memory as plain DFS
- – Extra LB computation per child



Tabu Search Modification: Reactive Tenure

Problem

Fixed tenure τ may be too short (cycling) or too long (slow exploration).

Solution: Dynamic Tenure

Adapt τ based on search history using solution hashing:

- If a solution is revisited (hash collision detected):

$$\tau \leftarrow \tau \times 1.1 \qquad \text{(increase to escape cycle)}$$

- Otherwise, slowly decay toward baseline:

$$\tau \leftarrow \tau \times 0.995 \qquad \text{(allow exploitation)}$$

Baseline: $\tau_0 = \lfloor n/4 \rfloor$. **Bounds:** $\tau \in [1, n]$.

Battiti & Tecchiolli, *ORSA J. Comp.*, 1994

Tabu Search Modification: Diversification Restarts

Problem

Search may stagnate in a region with no improving moves for many iterations.

Solution: Perturbation-Based Restart

After $10n$ non-improving iterations:

1. Return to the best-known solution π^*
2. Apply 3 random swaps to perturb it
3. Resume tabu search from the perturbed solution
4. Reset the tabu list

Combined with reactive tenure $\Rightarrow$ more robust exploration.
This is an instance of the *Iterated Local Search* (ILS) paradigm.

Stützle, 2006; Lourenço et al., ILS+TS

Experimental Setup

Datasets

- **QAPLIB** benchmark instances
- Families: `nug`, `chr`, `tai`
- $n \in \{12, 14, 15, 20, 25, 30\}$
- Best-known solutions (BKS) from QAPLIB

Metrics

- Runtime (seconds)
- Memory usage (MB)
- Nodes explored (B&B)
- Solution quality (Gap%)

Protocol

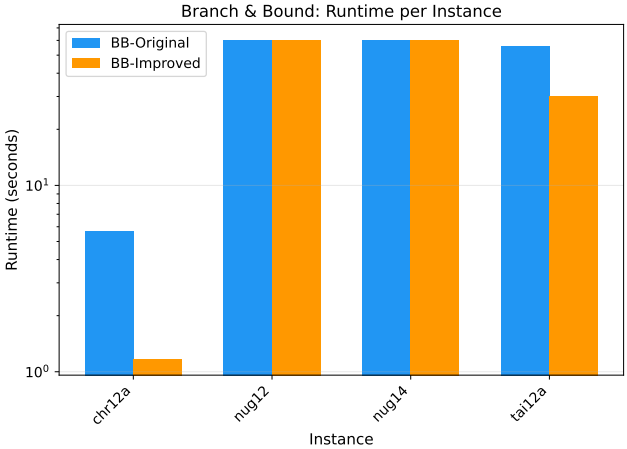
- B&B: $n \leq 14$ with 60 s timeout
- Tabu Search: all sizes, $T = \max(3000, 150n)$

Environment

- Hardware: standard laptop
- Python 3 with NumPy, SciPy
- Plots via Matplotlib

QAPLIB: Burkard, Karisch & Rendl, 1997

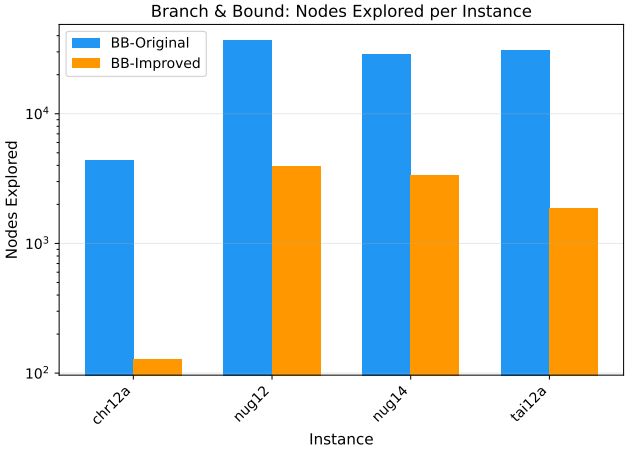
B&B Runtime Comparison



Observations

- Log-scale y-axis
- Improved B&B (greedy UB + LB-ordered DFS) shows faster convergence
- Greedy initialization provides a tighter starting bound

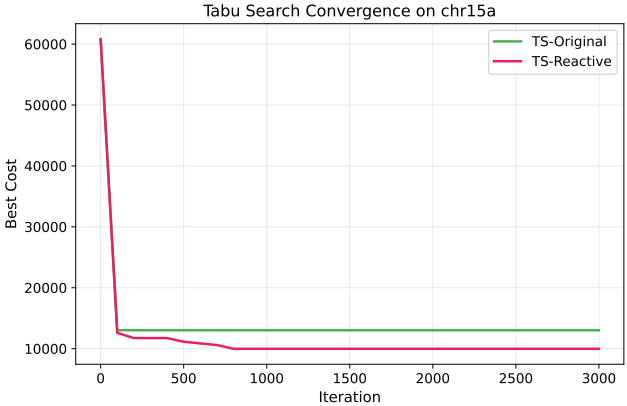
B&B Nodes Explored



Observations

- Greedy upper bound dramatically reduces nodes explored
- LB-ordered child expansion focuses DFS on promising branches
- Combined effect: 9–34× fewer nodes explored

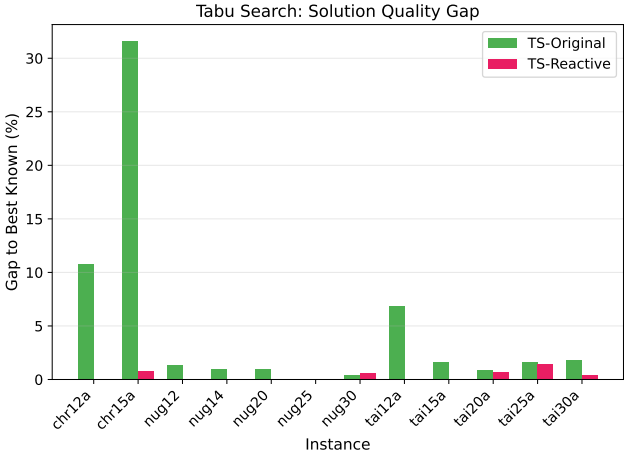
Tabu Search Convergence



Observations (chr15a)

- TS-Original stagnates at iteration 100
- TS-Reactive keeps improving via dynamic tenure and diversification
- Final gap: TS-Original 31.6% vs TS-Reactive 0.83%

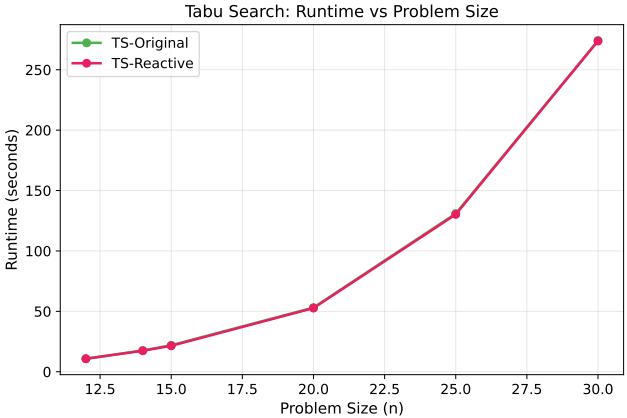
Tabu Search — Gap Comparison



Observations

- Gap% = $\frac{\text{found} - \text{best known}}{\text{best known}} \times 100$
- Reactive TS achieves lower gap% on most instances
- Largest improvement: chr15a (31.6% $\rightarrow$ 0.83%); consistently better at $n \leq 20$

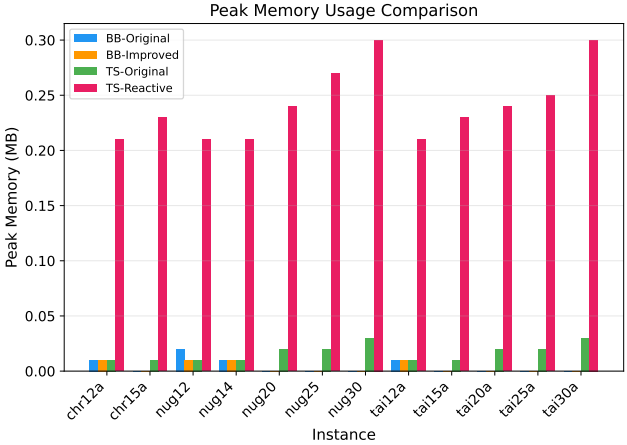
Tabu Search — Runtime Scalability



Observations

- TS scales from $n = 12$ to $n = 30$ without timeout
- Runtime grows as $\mathcal{O}(n^3)$–$\mathcal{O}(n^4)$: $n{=}12 \rightarrow 10\,\mathrm{s}$, $n{=}30 \rightarrow 274\,\mathrm{s}$
- TS-Reactive overhead is negligible ($<1\%$ vs Original)
- Unlike B&B, TS completes all instances

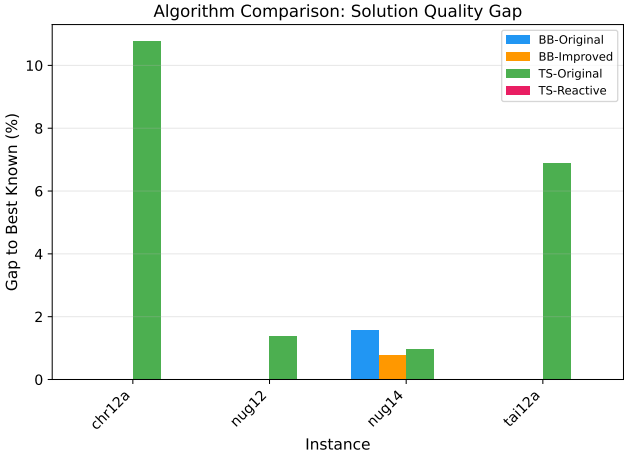
Memory Usage Comparison



Observations

- TS-Reactive uses most memory due to visited-solution hash set (up to 0.30 MB)
- B&B variants use negligible memory (<0.02 MB) with DFS stack
- Tabu Search is memory-efficient: $\mathcal{O}(n^2)$ for matrices + $\mathcal{O}(n)$ for tabu list
- Trade-off: memory vs. optimality guarantee

Overall Algorithm Comparison



Observations

- TS-Reactive finds optimal on all 4 instances; B&B on 3 of 4 (nug14 timeout)
- B&B hits timeout for $n \geq 12$; TS scales to $n = 30$

Results Summary

			B&B Orig.		B&B Impr.		TS React.
Instance	n	**BKS**	Cost	Time	Cost	Time	Gap %
nug12	12	578	**578**	T/O	**578**	T/O	**0.00**
chr12a	12	9552	**9552**	5.7	**9552**	1.2	**0.00**
tai12a	12	224416	**224416**	56.1	**224416**	30.0	**0.00**
nug14	14	1014	1030	T/O	**1022**	T/O	**0.00**
chr15a	15	9896	—	—	—	—	0.83
tai15a	15	388214	—	—	—	—	**0.00**
nug20	20	2570	—	—	—	—	**0.00**
tai20a	20	703482	—	—	—	—	0.67
nug25	25	3744	—	—	—	—	**0.00**
tai25a	25	1167256	—	—	—	—	1.43
nug30	30	6124	—	—	—	—	0.62
tai30a	30	1818146	—	—	—	—	0.39

T/O = timeout (60 s). Bold = optimal found. BKS from QAPLIB (Burkard et al., 1997).

B&B Improved: greedy UB + LB-ordered DFS. On chr12a: 5× speedup (5.7 s → 1.2 s); on nug14: better solution under timeout (1022 vs 1030).

TS Reactive finds BKS on 7 of 12 instances.

Real-World Applications

Facility & Campus Layout

F_{ij}: material flow
$D_{k\ell}$: physical distance

Minimize transport cost between departments.

VLSI & Chip Design

F_{ij}: net connectivity
$D_{k\ell}$: Manhattan distance

Minimize total wire length on chip.

Keyboard Layout

F_{ij}: digram frequency
$D_{k\ell}$: finger travel distance

Minimize total typing effort.

Reproducibility

Code Repository

All source code available at
`https://github.com/Tausif-Rashid/CSE462_Algorithm_Project`

Master Script

```
$ python main.py
```

- Uses QAPLIB benchmark instances
- Runs all algorithm variants (B&B original, B&B improved, TS original, TS reactive)
- Collects timing and quality metrics
- Produces all plots as PDF in `plots/`

Requirements

Python 3, NumPy, SciPy, Matplotlib

Key Takeaways

1. **QAP is NP-Hard** — proved via reductions from both Symmetric TSP and Balanced Cut
2. **Branch-and-Bound** gives optimal solutions but is limited to small instances ($n \leq 14$ within 60 s)
3. **Tabu Search** scales well to $n = 30+$; reactive tenure and diversification restarts improve solution quality
4. **Proposed modifications** show measurable improvement:
   - Greedy UB + LB-ordered DFS: 9–34× fewer nodes, 5× speedup on chr12a
   - Better solution under timeout on nug14 (1022 vs 1030)
   - Reactive tenure: lower gap% on most instances

References

1. Koopmans, T.C. & Beckmann, M. (1957). Assignment problems and the location of economic activities. *Econometrica*, 25(1), 53–76.
2. Gilmore, P.C. (1962). Optimal and suboptimal algorithms for the quadratic assignment problem. *SIAM Journal*, 10(2), 305–313.
3. Lawler, E.L. (1963). The quadratic assignment problem. *Management Science*, 9(4), 586–599.
4. Glover, F. (1989, 1990). Tabu search. *ORSA Journal on Computing*, 1(3), 190–206; 2(1), 4–32.
5. Taillard, É. (1991). Robust taboo search for the QAP. *Parallel Computing*, 17, 443–455.
6. Battiti, R. & Tecchiolli, G. (1994). The reactive tabu search. *ORSA Journal on Computing*, 6(2), 126–140.
7. Burkard, R.E., Karisch, S.E. & Rendl, F. (1997). QAPLIB — A quadratic assignment problem library. *JGLO*, 10, 391–403.
8. Loiola, E.M. et al. (2007). A survey for the quadratic assignment problem. *EJOR*, 176(2), 657–690.

Thank You!

Questions?

Md. Jakaria Hossain Musa Tur Farazi Md. As-Aid Rahman
MD Sajid Mostafiz Noor Tausif Rashid Awesh Islam

CSE 462 — Algorithm Engineering Lab
March 2026